CENTRO UNIVERSITÁRIO FILADÉLFIA

LUCAS LEONARDO SAMBATI

PETAGENDA:

SISTEMA WEB DE AGENDAMENTO PARA PET SHOP

LONDRINA
2026

LUCAS LEONARDO SAMBATI

PETAGENDA:

SISTEMA WEB DE AGENDAMENTO PARA PET SHOP

Trabalho apresentado à disciplina de Projeto de Desenvolvimento de Software I do curso de Engenharia de Software do Centro Universitário Filadélfia – UniFil, como requisito parcial de avaliação.

Professor: Fernando Nakagawa

LONDRINA
2026

LISTA DE FIGURAS

Figura 1 – Arquitetura do sistema PetAgenda (padrão MVC) 10
Figura 2 – Diagrama de casos de uso do PetAgenda 14
Figura 3 – Diagrama Entidade-Relacionamento do PetAgenda (modelo lógico) 18
Figura 4 – Diagrama de atividade do processo de agendamento 21
Figura 5 – Diagrama de classes das entidades do PetAgenda 22
Figura 6 – Diagrama de sequência do UC01 – Autenticar usuário 24
Figura 7 – Diagrama de sequência do UC02 – Manter clientes 25
Figura 8 – Diagrama de sequência do UC03 – Manter pets 26
Figura 9 – Diagrama de sequência do UC04 – Manter serviços 27
Figura 10 – Diagrama de sequência do UC05 – Manter agendamentos (parte 1) 28
Figura 11 – Diagrama de sequência do UC05 – Manter agendamentos (parte 2) 29
Figura 12 – Tela de login 30
Figura 13 – Painel inicial com a agenda do dia 31
Figura 14 – Agenda com filtros por data e status 31
Figura 15 – Cadastro de agendamento com a validação de conflito de horário (RN04) 32

LISTA DE QUADROS

Quadro 1 – Tecnologias utilizadas no projeto 8
Quadro 2 – Requisitos funcionais..... 10
Quadro 3 – Requisitos não funcionais..... 12
Quadro 4 – Regras de negócio 13
Quadro 5 – Especificação do caso de uso UC01 – Autenticar usuário 14
Quadro 6 – Especificação do caso de uso UC02 – Manter clientes..... 15
Quadro 7 – Especificação do caso de uso UC03 – Manter pets 16
Quadro 8 – Especificação do caso de uso UC04 – Manter serviços..... 16
Quadro 9 – Especificação do caso de uso UC05 – Manter agendamentos 17
Quadro 10 – Descrição das tabelas do banco de dados..... 19
Quadro 11 – Rastreabilidade entre os casos de uso e a implementação 32

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
CPF	Cadastro de Pessoas Físicas
CRUD	*Create, Read, Update, Delete* (cadastrar, consultar, alterar e excluir)
CSRF	*Cross-Site Request Forgery* (falsificação de solicitação entre sites)
CSS	*Cascading Style Sheets*
DER	Diagrama Entidade-Relacionamento
FK	*Foreign Key* (chave estrangeira)
HTML	*HyperText Markup Language*
HTTP	*HyperText Transfer Protocol*
MVC	*Model-View-Controller*
ORM	*Object-Relational Mapping* (mapeamento objeto-relacional)
PK	*Primary Key* (chave primária)
RF	Requisito Funcional
RN	Regra de Negócio
RNF	Requisito Não Funcional
SGBD	Sistema Gerenciador de Banco de Dados
SQL	*Structured Query Language*
UC	*Use Case* (caso de uso)
UML	*Unified Modeling Language* (Linguagem de Modelagem Unificada)

SUMÁRIO

1 INTRODUÇÃO 6
1.1 OBJETIVOS 6
1.1.1 Objetivo geral 6
1.1.2 Objetivos específicos 6
1.2 JUSTIFICATIVA 7
1.3 TECNOLOGIAS UTILIZADAS 7
1.4 ORGANIZAÇÃO DO TRABALHO 8
2 DESENVOLVIMENTO 9
2.1 VISÃO GERAL DO SISTEMA 9
2.1.1 Arquitetura 9
2.2 REQUISITOS FUNCIONAIS 10
2.3 REQUISITOS NÃO FUNCIONAIS 12
2.4 REGRAS DE NEGÓCIO 13
2.5 DIAGRAMA DE CASOS DE USO 14
2.5.1 Especificação dos casos de uso 14
2.6 DIAGRAMA ENTIDADE-RELACIONAMENTO (DER) 18
2.7 DIAGRAMA DE ATIVIDADE 20
2.8 DIAGRAMA DE CLASSES 22
2.9 DIAGRAMAS DE SEQUÊNCIA 23
2.9.1 UC01 – Autenticar usuário 23
2.9.2 UC02 – Manter clientes 25
2.9.3 UC03 – Manter pets 26
2.9.4 UC04 – Manter serviços 27
2.9.5 UC05 – Manter agendamentos 28
2.10 PROTÓTIPOS DAS INTERFACES 30
2.11 IMPLEMENTAÇÃO E REPOSITÓRIO 32
3 CONCLUSÃO 34
REFERÊNCIAS 36
APÊNDICE A – SCRIPT DE CRIAÇÃO DO BANCO DE DADOS 38

1 INTRODUÇÃO

Os pet shops de bairro realizam diariamente diversos atendimentos de banho e tosa. Em estabelecimentos de pequeno porte, é comum que a agenda seja controlada em cadernos, planilhas ou conversas em aplicativos de mensagens. Essa forma de controle favorece erros como horários marcados em duplicidade, dados de clientes espalhados em diferentes lugares, dificuldade para consultar o histórico de cada animal e falta de padronização nas informações registradas pela equipe.

Diante desse cenário, este trabalho apresenta o **PetAgenda**, um sistema web para o agendamento de serviços de banho e tosa em pet shops de pequeno porte. O sistema permite cadastrar os clientes (tutores), os seus pets e os serviços oferecidos, além de registrar e acompanhar os agendamentos, aplicando automaticamente regras como o horário de funcionamento do estabelecimento, o impedimento de conflitos de horário e o cálculo do horário de término de cada atendimento. Todas as informações são armazenadas em um banco de dados relacional e manipuladas por meio das operações de cadastro, consulta, alteração e exclusão, conhecidas pela sigla CRUD (*Create, Read, Update e Delete*).

O projeto foi elaborado na disciplina de Projeto de Desenvolvimento de Software I, cujo foco é a documentação do sistema, e implementado na disciplina de Projeto de Desenvolvimento de Software II. A documentação segue as boas práticas da Engenharia de Software (SOMMERVILLE, 2018; PRESSMAN; MAXIM, 2021) e utiliza a UML (*Unified Modeling Language*) como linguagem de modelagem (OBJECT MANAGEMENT GROUP, 2017).

1.1 OBJETIVOS

1.1.1 Objetivo geral

Projetar, documentar e implementar um sistema web que informatize o agendamento de serviços de banho e tosa de um pet shop de pequeno porte, com persistência das informações em banco de dados relacional.

1.1.2 Objetivos específicos

Para atingir o objetivo geral, foram definidos os seguintes objetivos específicos:

a) levantar e especificar os requisitos funcionais, os requisitos não funcionais e as regras de negócio do sistema;
b) modelar as funcionalidades com o diagrama de casos de uso e descrever cada caso de uso;
c) modelar o banco de dados por meio do Diagrama Entidade-Relacionamento (DER);
d) representar o processo de agendamento com o diagrama de atividade e a estrutura do domínio com o diagrama de classes;
e) detalhar a interação entre os componentes de cada caso de uso com diagramas de sequência;
f) implementar as operações de CRUD com persistência em banco de dados e testes automatizados;
g) versionar o código-fonte em um repositório público no GitHub.

1.2 JUSTIFICATIVA

A informatização da agenda reduz falhas operacionais simples, mas custosas, como o atendimento de dois animais no mesmo horário ou a perda do contato de um cliente. Ao centralizar os dados de clientes, pets, serviços e agendamentos, o sistema facilita a consulta do histórico de atendimentos, padroniza o registro das informações e oferece ao proprietário uma visão diária do movimento do estabelecimento.

Do ponto de vista acadêmico, o projeto permite aplicar, em um problema real e de escopo controlado, os conteúdos de engenharia de requisitos, modelagem UML, projeto de banco de dados e desenvolvimento web estudados no curso de Engenharia de Software.

1.3 TECNOLOGIAS UTILIZADAS

O sistema foi desenvolvido como uma aplicação web em Python (PYTHON SOFTWARE FOUNDATION, 2025), utilizando o *microframework* Flask (PALLETS, 2025) e o mapeamento objeto-relacional (ORM) do SQLAlchemy (SQLALCHEMY, 2025) para acesso ao banco de dados SQLite (SQLITE, 2025). A escolha dessa pilha tecnológica (*stack*) considerou a simplicidade de instalação, já que o SQLite armazena o banco em um único arquivo e dispensa um servidor de banco de dados, a

legibilidade da linguagem Python e a ampla documentação disponível. O Quadro 1 resume as tecnologias empregadas.

Quadro 1 – Tecnologias utilizadas no projeto

Tecnologia	**Versão**	**Finalidade no projeto**
Python	3.13	Linguagem de programação da aplicação (*back-end*).
Flask	3.1	*Microframework* web: rotas (controladores), sessões e integração com os templates.
SQLAlchemy e Flask-SQLAlchemy	2.1 e 3.1	Mapeamento objeto-relacional (ORM) e acesso ao banco de dados.
SQLite	3	Sistema gerenciador de banco de dados relacional, armazenado em arquivo.
Jinja2	3.1	Motor de templates que gera as páginas HTML (camada View).
HTML5, CSS3, JavaScript e Bootstrap	5.3 (Bootstrap)	Construção da interface responsiva.
Werkzeug	3.1	Geração e verificação do *hash* das senhas.
pytest	9.2	Testes automatizados.
Git e GitHub	—	Controle de versão e hospedagem do código-fonte.
PlantUML	1.2026	Elaboração dos diagramas UML e do DER.
Visual Studio Code	—	Editor de código utilizado no desenvolvimento.

Fonte: elaborado pelo autor (2026).

1.4 ORGANIZAÇÃO DO TRABALHO

Este projeto foi desenvolvido individualmente pelo aluno Lucas Leonardo Sambati. Além desta introdução, o trabalho está organizado da seguinte forma: o capítulo 2 apresenta o desenvolvimento, com a visão geral e a arquitetura do sistema, os requisitos funcionais e não funcionais, as regras de negócio, os diagramas de casos de uso, entidade-relacionamento, atividade, classes e sequência, os protótipos das interfaces e a implementação; o capítulo 3 apresenta a conclusão, com os aprendizados obtidos; por fim, são apresentadas as referências e, no Apêndice A, o script de criação do banco de dados. A formatação segue a norma NBR 14724 (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2011).

2 DESENVOLVIMENTO

Este capítulo apresenta a especificação e a modelagem do PetAgenda. Inicialmente, são descritos o escopo e a arquitetura do sistema; em seguida, os requisitos e as regras de negócio; depois, os diagramas UML e o modelo de dados; por fim, as interfaces e a organização da implementação.

2.1 VISÃO GERAL DO SISTEMA

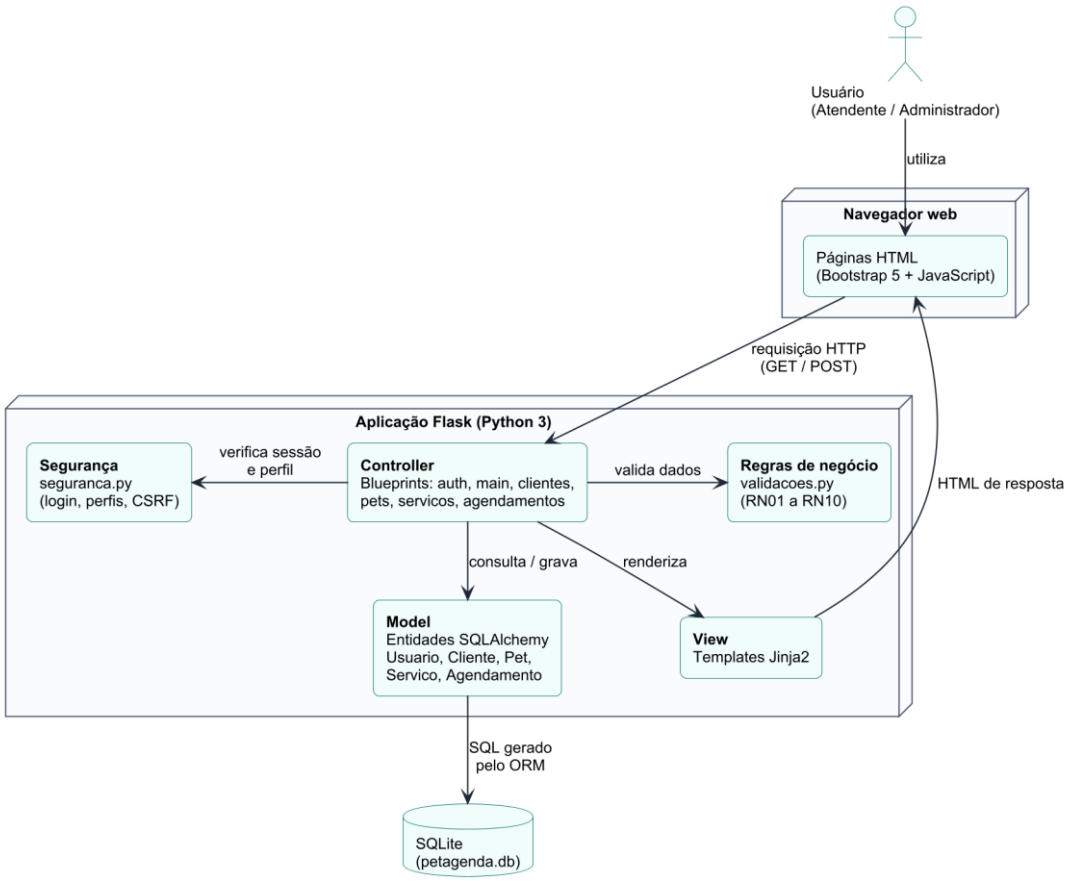
O PetAgenda é destinado aos funcionários de um pet shop de pequeno porte que possui uma estação de banho e tosa. O sistema possui dois perfis de acesso: o **Atendente**, responsável pelo atendimento diário (cadastro de clientes e pets e controle da agenda), e o **Administrador**, normalmente o proprietário, que possui todas as permissões do atendente e, adicionalmente, gerencia o catálogo de serviços, com seus preços e durações.

O escopo do sistema compreende: autenticação de usuários; cadastro e consulta de clientes, pets e serviços; agendamento, remarcação, conclusão, cancelamento e exclusão de atendimentos; e um painel inicial com a agenda do dia. Ficam fora do escopo desta versão o agendamento feito diretamente pelo cliente, o envio de lembretes por mensagem, o controle financeiro e de pagamentos e o gerenciamento de várias estações de atendimento simultâneas, apontados como trabalhos futuros.

2.1.1 Arquitetura

A aplicação segue o padrão arquitetural MVC (*Model-View-Controller*), que separa o acesso aos dados (Model), a apresentação (View) e o tratamento das requisições (Controller) (PRESSMAN; MAXIM, 2021). No PetAgenda, o Model é formado pelas classes de entidade mapeadas pelo SQLAlchemy; a View, pelos templates Jinja2 que geram as páginas HTML; e o Controller, pelos *blueprints* do Flask, um para cada caso de uso. As regras de negócio ficam concentradas no módulo de validações, e o controle de acesso, no módulo de segurança, conforme ilustra a Figura 1.

Figura 1 – Arquitetura do sistema PetAgenda (padrão MVC)



Fonte: elaborado pelo autor (2026).

2.2 REQUISITOS FUNCIONAIS

Requisitos funcionais são declarações dos serviços que o sistema deve fornecer, de como ele deve reagir a determinadas entradas e de como deve se comportar em situações específicas (SOMMERVILLE, 2018). O Quadro 2 apresenta os requisitos funcionais do PetAgenda, com a prioridade de cada um e o caso de uso (UC) ao qual pertencem.

Quadro 2 – Requisitos funcionais

ID	Requisito	Descrição	Prioridade	UC
RF01	Autenticar usuário	Permitir o acesso ao sistema somente mediante e-mail e senha válidos de um usuário ativo.	Alta	UC01
RF02	Encerrar sessão	Permitir que o usuário saia do sistema (*logout*), encerrando a sessão.	Alta	UC01
RF03	Cadastrar cliente	Registrar nome, CPF, telefone, e-mail e endereço do cliente (tutor).	Alta	UC02

ID	Requisito	Descrição	Prioridade	UC
RF04	Consultar clientes	Listar os clientes em ordem alfabética, com busca por nome, CPF ou telefone, e exibir os dados e os pets de um cliente.	Alta	UC02
RF05	Alterar cliente	Permitir a alteração dos dados cadastrais de um cliente.	Alta	UC02
RF06	Excluir cliente	Permitir a exclusão de um cliente, mediante confirmação.	Média	UC02
RF07	Cadastrar pet	Registrar nome, espécie, raça, porte, data de nascimento e observações do pet, vinculado a um cliente.	Alta	UC03
RF08	Consultar pets	Listar os pets, com busca pelo nome do pet ou do tutor, exibindo a idade calculada.	Alta	UC03
RF09	Alterar pet	Permitir a alteração dos dados do pet, inclusive a troca de tutor.	Alta	UC03
RF10	Excluir pet	Permitir a exclusão de um pet, mediante confirmação.	Média	UC03
RF11	Cadastrar serviço	Registrar nome, descrição, preço, duração em minutos e situação (ativo ou inativo) do serviço.	Alta	UC04
RF12	Consultar serviços	Listar os serviços com preço, duração e situação.	Alta	UC04
RF13	Alterar serviço	Permitir a alteração dos dados do serviço, inclusive a sua inativação.	Alta	UC04
RF14	Excluir serviço	Permitir a exclusão de um serviço, mediante confirmação.	Média	UC04
RF15	Cadastrar agendamento	Registrar pet, serviço, data, horário de início e observações, calculando automaticamente o horário de término e o valor.	Alta	UC05
RF16	Consultar agenda	Listar os agendamentos com filtros por data e por status, exibindo o total previsto.	Alta	UC05
RF17	Alterar agendamento	Permitir remarcar a data e o horário ou trocar o pet ou o serviço de um agendamento.	Alta	UC05
RF18	Alterar status do agendamento	Permitir marcar um agendamento como concluído ou cancelado.	Alta	UC05
RF19	Excluir agendamento	Permitir a exclusão de um agendamento, mediante confirmação.	Média	UC05
RF20	Exibir painel inicial	Exibir os totais de clientes, pets, serviços ativos e agendamentos do dia, a agenda de hoje e os próximos agendamentos.	Média	UC05

Fonte: elaborado pelo autor (2026).

2.3 REQUISITOS NÃO FUNCIONAIS

Requisitos não funcionais são restrições sobre os serviços ou as funções oferecidos pelo sistema, como restrições de tempo, de segurança e de padrões a serem seguidos, e frequentemente se aplicam ao sistema como um todo (SOMMERVILLE, 2018). Os requisitos do Quadro 3 foram classificados de acordo com as características de qualidade de produto definidas pela norma ISO/IEC 25010 (INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 2023).

Quadro 3 – Requisitos não funcionais

ID	**Categoria**	**Descrição**
RNF01	Segurança	As senhas devem ser armazenadas somente na forma de *hash*, gerado por algoritmo seguro (scrypt) da biblioteca Werkzeug, nunca em texto puro.
RNF02	Segurança	Todas as funcionalidades, exceto a tela de login, devem exigir usuário autenticado; o cadastro, a alteração e a exclusão de serviços devem ser restritos ao perfil Administrador.
RNF03	Segurança	Os formulários devem ser protegidos contra ataques CSRF por meio de *token* vinculado à sessão, e as consultas ao banco devem ser parametrizadas pelo ORM, evitando injeção de SQL.
RNF04	Usabilidade	A interface deve ser responsiva, em português do Brasil, com mensagens claras de sucesso e de erro, destaque dos campos inválidos e confirmação antes de exclusões e cancelamentos.
RNF05	Desempenho	As operações de consulta e de gravação devem responder em até 2 segundos em condições normais de uso, com até 10 mil registros por tabela.
RNF06	Confiabilidade	A integridade dos dados deve ser garantida no banco por chaves primárias e estrangeiras, restrições UNIQUE, NOT NULL e CHECK, além da validação dos dados no servidor.
RNF07	Portabilidade	O sistema deve executar em Windows, Linux e macOS com Python 3.11 ou superior, sem necessidade de instalar um servidor de banco de dados.
RNF08	Compatibilidade	A interface deve funcionar nas versões atuais dos navegadores Google Chrome, Microsoft Edge e Mozilla Firefox.
RNF09	Manutenibilidade	O código deve seguir a arquitetura MVC, com um controlador por caso de uso, o guia de estilo PEP 8 e testes automatizados das regras de negócio e das operações de CRUD.
RNF10	Implementação	O sistema deve ser desenvolvido em Python com o *framework* Flask, o ORM SQLAlchemy e o banco de dados SQLite, com o código versionado no GitHub.

Fonte: elaborado pelo autor (2026).

2.4 REGRAS DE NEGÓCIO

As regras de negócio expressam políticas e restrições do domínio que o sistema deve respeitar, independentemente da tecnologia utilizada. Elas foram identificadas a partir da rotina de um pet shop, estão implementadas no módulo de validações e nas classes de entidade e são verificadas pelos testes automatizados. O Quadro 4 apresenta as regras do PetAgenda.

Quadro 4 – Regras de negócio

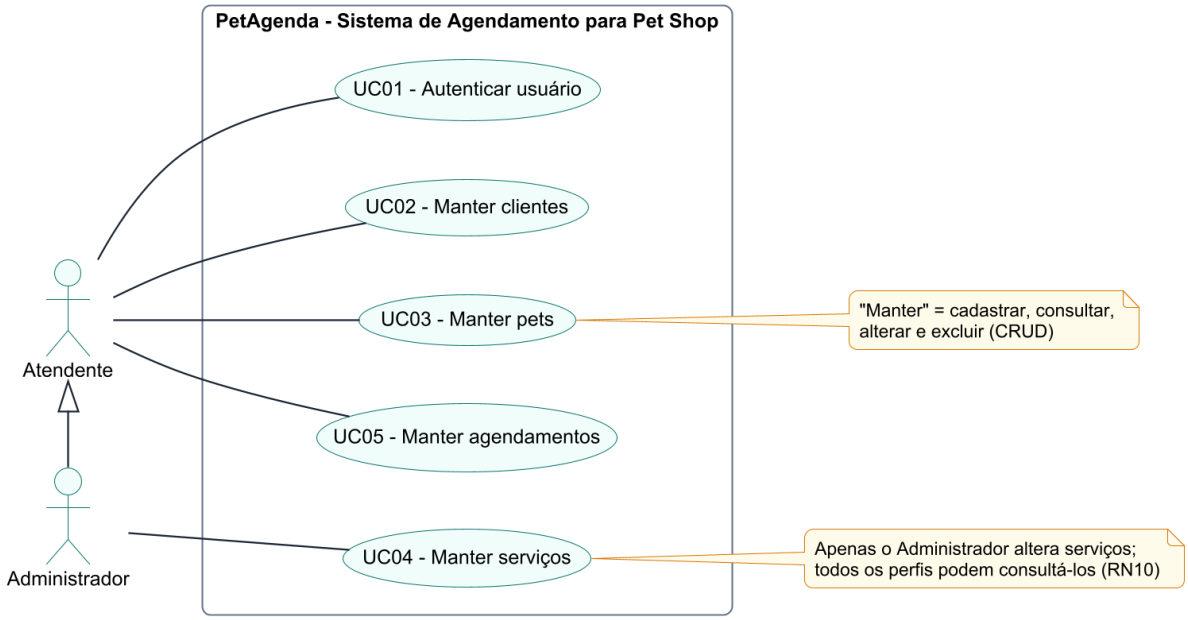
ID	Regra	Descrição
RN01	CPF válido e único	O CPF do cliente deve possuir dígitos verificadores válidos e não pode se repetir entre clientes.
RN02	Pet vinculado a um tutor	Todo pet deve pertencer a exatamente um cliente cadastrado.
RN03	Agendamento futuro	Não é permitido agendar em data ou horário que já passou.
RN04	Sem conflito de horário	O pet shop possui uma estação de banho e tosa; por isso, dois agendamentos com status “Agendado” não podem ter períodos sobrepostos.
RN05	Horário de funcionamento	Os atendimentos ocorrem de segunda a sábado, das 08h às 18h; o serviço deve começar e terminar dentro desse intervalo.
RN06	Término e valor automáticos	O horário de término é calculado pela duração do serviço, e o valor do agendamento é o preço do serviço no momento do agendamento; alterações posteriores de preço não modificam agendamentos existentes.
RN07	Ciclo de vida do agendamento	Todo agendamento é criado com status “Agendado” e pode passar para “Concluído” ou “Cancelado”; agendamentos concluídos ou cancelados não podem ser alterados. O cancelamento libera o horário.
RN08	Preservação do histórico	Não é permitido excluir cliente que possua pets nem pet que possua agendamentos.
RN09	Serviço inativo	Serviços inativos não podem receber novos agendamentos; serviços já utilizados em agendamentos não podem ser excluídos, apenas inativados.
RN10	Perfis de acesso	Somente o Administrador pode cadastrar, alterar e excluir serviços; os demais perfis apenas os consultam.

Fonte: elaborado pelo autor (2026).

2.5 DIAGRAMA DE CASOS DE USO

O diagrama de casos de uso representa as funcionalidades do sistema do ponto de vista dos seus usuários, chamados de atores, e é utilizado principalmente nas etapas de levantamento e análise de requisitos (GUEDES, 2018). A Figura 2 apresenta os cinco casos de uso do PetAgenda. O ator Administrador é uma especialização do ator Atendente (generalização entre atores): ele herda o acesso a todos os casos de uso do atendente e, além disso, é o único que pode manter os serviços. Os casos de uso denominados “Manter” agrupam as quatro operações do CRUD (cadastrar, consultar, alterar e excluir) sobre a respectiva entidade.

Figura 2 – Diagrama de casos de uso do PetAgenda



Fonte: elaborado pelo autor (2026).

2.5.1 Especificação dos casos de uso

Os quadros a seguir descrevem cada caso de uso, com os atores, as pré-condições, o fluxo principal, os fluxos alternativos e de exceção, as pós-condições e os requisitos e regras relacionados.

Quadro 5 – Especificação do caso de uso UC01 – Autenticar usuário

Caso de uso	UC01 – Autenticar usuário
Atores	Atendente e Administrador
Objetivo	Permitir que o funcionário acesse o sistema com segurança e encerre a sua sessão ao final do uso.

Pré-condição	O usuário possui cadastro ativo no sistema.
Fluxo principal	1. O usuário acessa o endereço do sistema. 2. O sistema exibe a tela de login. 3. O usuário informa o e-mail e a senha e aciona “Entrar”. 4. O sistema localiza o usuário pelo e-mail e compara a senha informada com o *hash* armazenado. 5. O sistema cria a sessão e exibe o painel inicial com a mensagem de boas-vindas.
Fluxos alternativos e de exceção	A1 – Credenciais inválidas (passo 4): o sistema exibe “E-mail ou senha inválidos.” e permanece na tela de login. A2 – Usuário inativo (passo 4): o acesso é negado com a mesma mensagem genérica. A3 – Encerrar sessão: a qualquer momento, o usuário aciona “Sair”; o sistema encerra a sessão e retorna à tela de login.
Pós-condição	Usuário autenticado, com acesso às funcionalidades do seu perfil.
Requisitos e regras	RF01, RF02, RNF01, RNF02 e RNF03.

Fonte: elaborado pelo autor (2026).

Quadro 6 – Especificação do caso de uso UC02 – Manter clientes

Caso de uso	UC02 – Manter clientes
Atores	Atendente (e Administrador, por herança)
Objetivo	Cadastrar, consultar, alterar e excluir os clientes (tutores) do pet shop.
Pré-condição	Usuário autenticado.
Fluxo principal	1. O atendente acessa o menu “Clientes”, e o sistema lista os clientes em ordem alfabética. 2. O atendente aciona “Novo cliente”. 3. O atendente informa nome, CPF, telefone, e-mail e endereço e aciona “Salvar”. 4. O sistema valida os dados (RN01). 5. O sistema grava o cliente e exibe a página de detalhes com a mensagem de sucesso.
Fluxos alternativos e de exceção	A1 – Consultar: o atendente informa nome, CPF ou telefone, e o sistema exibe os clientes correspondentes; ao selecionar um cliente, são exibidos os seus dados e os seus pets. A2 – Alterar: o atendente aciona “Editar”, altera os dados e salva; o sistema valida e atualiza o registro. A3 – Excluir: o atendente aciona “Excluir” e confirma; se o cliente possuir pets, o sistema impede a exclusão e exibe um aviso (RN08). E1 – Dados inválidos (passo 4): CPF inválido ou já cadastrado, telefone incompleto ou nome ausente; o sistema destaca os campos com erro e não grava os dados.
Pós-condição	Cliente cadastrado, atualizado ou excluído do banco de dados.

Requisitos e regras	RF03, RF04, RF05, RF06, RN01 e RN08.

Fonte: elaborado pelo autor (2026).

Quadro 7 – Especificação do caso de uso UC03 – Manter pets

Caso de uso	UC03 – Manter pets
Atores	Atendente (e Administrador, por herança)
Objetivo	Cadastrar, consultar, alterar e excluir os pets, sempre vinculados a um cliente.
Pré-condição	Usuário autenticado e pelo menos um cliente cadastrado.
Fluxo principal	1. O atendente aciona “Novo pet” na lista de pets ou na página do cliente. 2. O sistema exibe o formulário com a lista de tutores. 3. O atendente seleciona o tutor e informa nome, espécie, porte, raça, data de nascimento e observações. 4. O sistema valida os dados e verifica se o tutor existe (RN02). 5. O sistema grava o pet e exibe a página do tutor com o pet cadastrado.
Fluxos alternativos e de exceção	A1 – Consultar: o atendente pesquisa pelo nome do pet ou do tutor; o sistema exibe espécie, raça, porte, idade calculada e tutor. A2 – Alterar: o atendente edita os dados do pet, podendo inclusive transferi-lo para outro tutor. A3 – Excluir: o atendente confirma a exclusão; se o pet possuir agendamentos, o sistema impede a operação para preservar o histórico (RN08). E1 – Dados inválidos (passo 4): campos obrigatórios ausentes ou data de nascimento futura; o sistema destaca os campos e não grava.
Pós-condição	Pet cadastrado, atualizado ou excluído.
Requisitos e regras	RF07, RF08, RF09, RF10, RN02 e RN08.

Fonte: elaborado pelo autor (2026).

Quadro 8 – Especificação do caso de uso UC04 – Manter serviços

Caso de uso	UC04 – Manter serviços
Atores	Administrador
Objetivo	Manter o catálogo de serviços do pet shop, com nome, descrição, preço, duração e situação.
Pré-condição	Usuário autenticado com o perfil Administrador.
Fluxo principal	1. O administrador acessa o menu “Serviços” e aciona “Novo serviço”. 2. O administrador informa nome, descrição, preço, duração (em minutos) e situação e aciona “Salvar”. 3. O sistema verifica o perfil do usuário (RN10) e valida os dados: nome único, preço maior que zero e duração múltipla de 15 minutos. 4. O sistema grava o serviço e exibe a lista atualizada.

Fluxos alternativos e de exceção	A1 – Consultar: qualquer usuário autenticado visualiza a lista de serviços, com preço, duração e situação. A2 – Alterar: o administrador altera os dados ou inativa o serviço, retirando-o de novos agendamentos (RN09). A3 – Excluir: o administrador confirma a exclusão; se o serviço já tiver sido agendado, o sistema impede a operação e orienta a inativá-lo (RN09). E1 – Acesso negado: um atendente que tente cadastrar, alterar ou excluir serviços recebe a página de acesso negado (HTTP 403). E2 – Dados inválidos (passo 3): o sistema destaca os campos com erro e não grava.
Pós-condição	Serviço cadastrado, atualizado, inativado ou excluído.
Requisitos e regras	RF11, RF12, RF13, RF14, RN09 e RN10.

Fonte: elaborado pelo autor (2026).

Quadro 9 – Especificação do caso de uso UC05 – Manter agendamentos

Caso de uso	UC05 – Manter agendamentos
Atores	Atendente (e Administrador, por herança)
Objetivo	Agendar, consultar, remarcar, concluir, cancelar e excluir os atendimentos de banho e tosa.
Pré-condição	Usuário autenticado; pet e serviço ativo cadastrados.
Fluxo principal	1. O atendente aciona “Novo agendamento”. 2. O atendente seleciona o pet, o serviço, a data e o horário de início e aciona “Confirmar agendamento”. 3. O sistema calcula o horário de término pela duração do serviço (RN06). 4. O sistema verifica se a data é futura (RN03), se o atendimento está dentro do horário de funcionamento (RN05) e se não há conflito com outro agendamento (RN04). 5. O sistema grava o agendamento com o status “Agendado”, o valor vigente do serviço e o usuário responsável. 6. O sistema exibe a agenda do dia com a confirmação.
Fluxos alternativos e de exceção	A1 – Consultar: o atendente filtra a agenda por data e por status; o painel inicial exibe a agenda do dia e os próximos agendamentos (RF20). A2 – Remarcar: o atendente altera pet, serviço, data ou horário de um agendamento com status “Agendado”; as regras do passo 4 são verificadas novamente (RN07). A3 – Concluir ou cancelar: o atendente altera o status; o cancelamento libera o horário para outro agendamento (RN07). A4 – Excluir: o atendente confirma a exclusão definitiva do agendamento. E1 – Regra violada (passo 4): o sistema informa o motivo (horário que já passou, fora do expediente ou indisponível) e não grava o agendamento. E2 – Serviço inativo: o sistema não permite agendar o serviço (RN09).
Pós-condição	Agendamento registrado, alterado ou excluído, com a agenda atualizada.

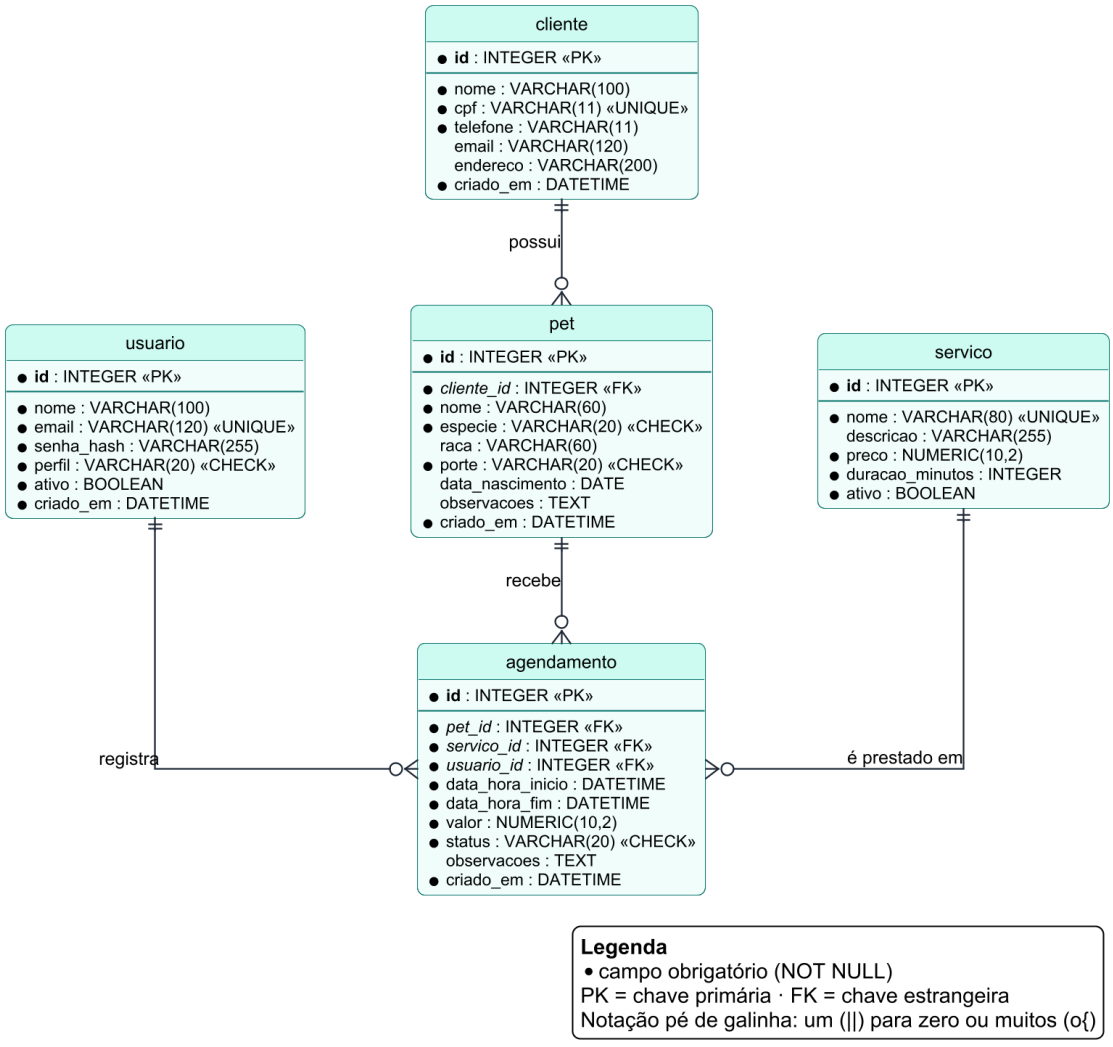
Requisitos e regras	RF15, RF16, RF17, RF18, RF19, RF20, RN03, RN04, RN05, RN06, RN07 e RN09.

Fonte: elaborado pelo autor (2026).

2.6 DIAGRAMA ENTIDADE-RELACIONAMENTO (DER)

O modelo de dados descreve a estrutura do banco de dados, identificando as entidades, os seus atributos e os relacionamentos entre elas (HEUSER, 2009). A Figura 3 apresenta o DER do PetAgenda no nível lógico, em notação pé de galinha (*Crow's Foot*), com as cinco tabelas do sistema, as chaves primárias (PK), as chaves estrangeiras (FK), os tipos de dados e as restrições de cada coluna.

Figura 3 – Diagrama Entidade-Relacionamento do PetAgenda (modelo lógico)



Fonte: elaborado pelo autor (2026).

Os relacionamentos do modelo são todos do tipo um-para-muitos (1:N):

a) **cliente possui pet**: um cliente pode ter zero ou muitos pets, e cada pet pertence a exatamente um cliente (chave estrangeira `pet.cliente_id`);

b) **pet recebe agendamento**: um pet pode ter zero ou muitos agendamentos, e cada agendamento refere-se a um único pet;

c) **serviço é prestado em agendamento**: um serviço pode estar em zero ou muitos agendamentos, e cada agendamento refere-se a um único serviço;

d) **usuário registra agendamento**: cada agendamento guarda o funcionário que o registrou, o que garante a rastreabilidade das operações.

A integridade dos dados é garantida no próprio banco por restrições de chave primária, chave estrangeira com ON DELETE RESTRICT, que impede a exclusão de registros que possuam dependentes (RN08), UNIQUE (CPF, e-mail e nome do serviço), NOT NULL e CHECK, que limita os valores permitidos para perfil, espécie, porte e status (ELMASRI; NAVATHE, 2018). Como o SQLite só aplica as chaves estrangeiras quando o comando PRAGMA foreign_keys está ativado (SQLITE, 2025), a aplicação habilita essa opção em todas as conexões. A tabela agendamento armazena o horário de término e o valor do serviço no momento do agendamento, o que simplifica a verificação de conflitos de horário (RN04) e preserva o histórico de preços (RN06). O Quadro 10 descreve as tabelas, e o script completo de criação do banco encontra-se no Apêndice A.

Quadro 10 – Descrição das tabelas do banco de dados

Tabela	**Descrição**	**Principais restrições**
usuario	Funcionários que acessam o sistema e os respectivos perfis.	PK id; UNIQUE email; CHECK perfil (ADMIN, ATENDENTE).
cliente	Tutores dos pets e os seus dados de contato.	PK id; UNIQUE cpf (11 dígitos).
pet	Animais atendidos, vinculados a um cliente.	PK id; FK cliente_id (RESTRICT); CHECK espécie e porte.
servico	Catálogo de serviços, com preço, duração e situação.	PK id; UNIQUE nome; preço NUMERIC(10,2).
agendamento	Reserva de um serviço para um pet em determinado período, com valor, status e usuário responsável.	PK id; FKs pet_id, servico_id e usuario_id (RESTRICT); CHECK status; índice em data_hora_inicio.

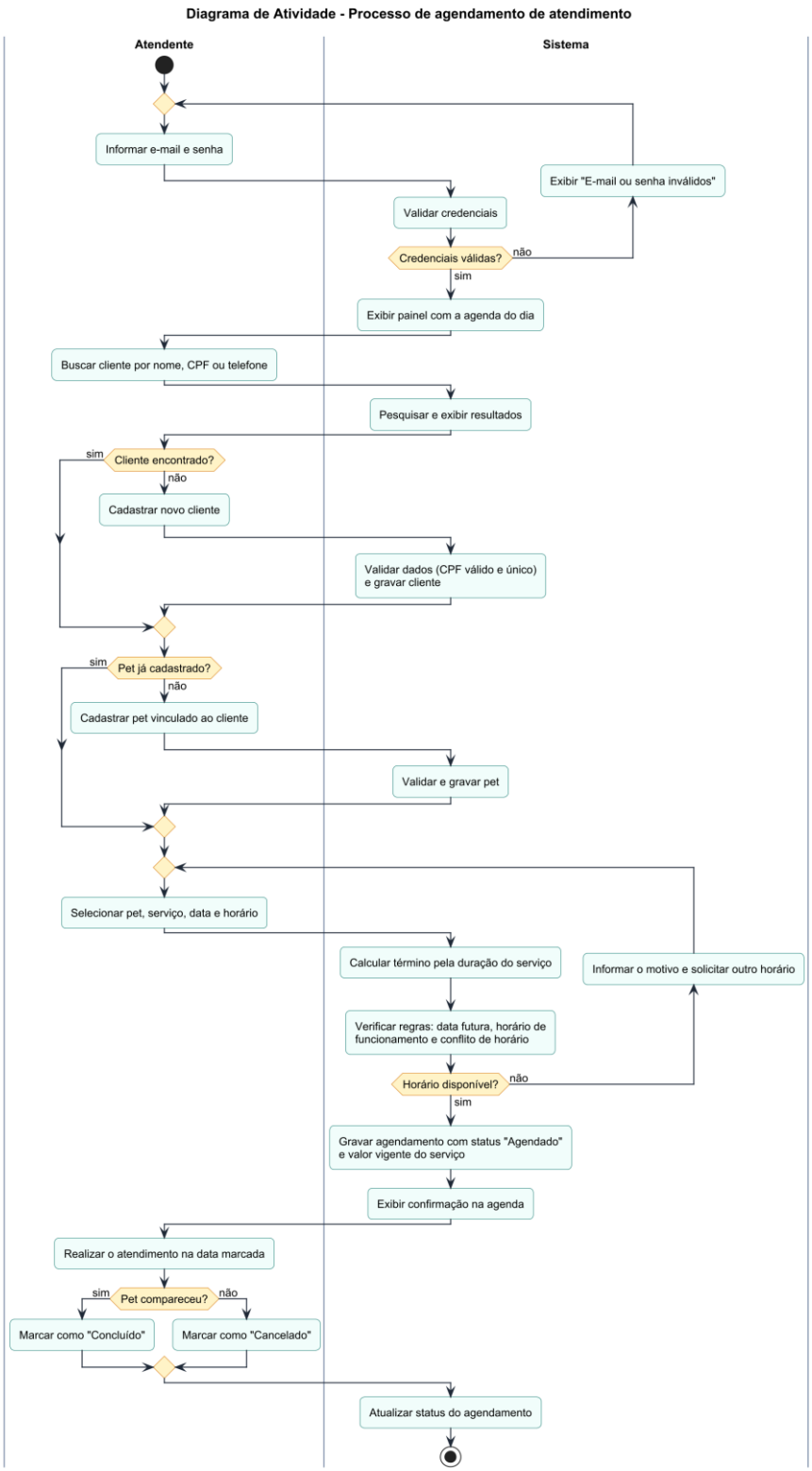
Fonte: elaborado pelo autor (2026).

2.7 DIAGRAMA DE ATIVIDADE

O diagrama de atividade descreve o fluxo de controle de um processo, destacando as ações, as decisões e a responsabilidade de cada participante por meio de raias (*swimlanes*) (BOOCH; RUMBAUGH; JACOBSON, 2012). A Figura 4 representa o principal processo de negócio do PetAgenda: o agendamento de um atendimento, desde a autenticação do atendente até a atualização do status após o atendimento.

O processo inicia com a autenticação, que se repete enquanto as credenciais forem inválidas. Em seguida, o atendente busca o cliente e, caso ele não exista, realiza o seu cadastro; o mesmo ocorre com o pet. Depois, o atendente escolhe o serviço, a data e o horário, e o sistema calcula o término do atendimento e verifica as regras de negócio. Se o horário não estiver disponível, o sistema informa o motivo e solicita outra escolha; caso contrário, grava o agendamento com o status “Agendado”. Na data marcada, o atendimento é concluído ou, se o pet não comparecer, cancelado.

Figura 4 – Diagrama de atividade do processo de agendamento

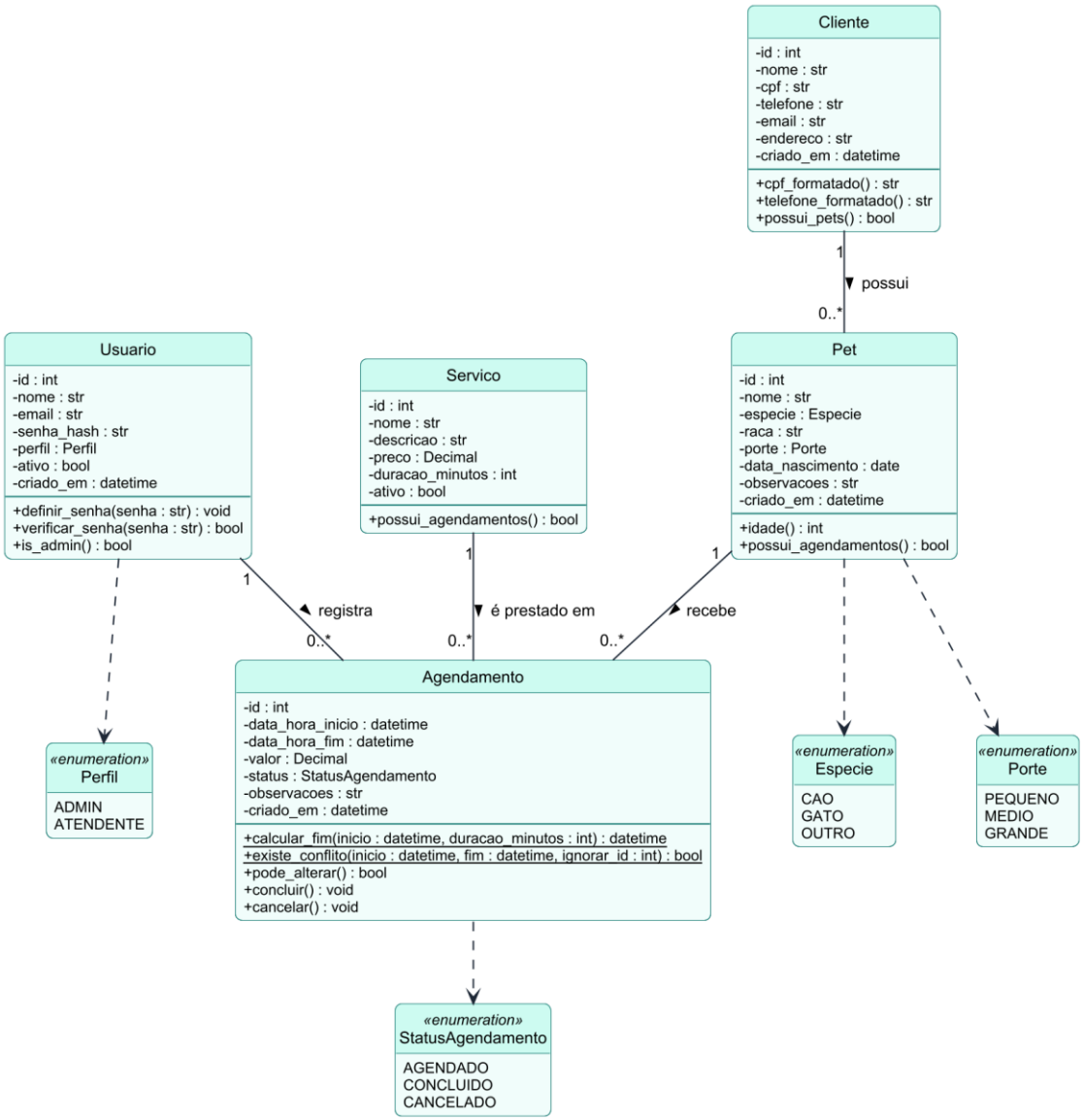


Fonte: elaborado pelo autor (2026).

2.8 DIAGRAMA DE CLASSES

O diagrama de classes apresenta a estrutura estática do sistema por meio das classes, dos seus atributos, das suas operações e dos relacionamentos entre elas (FOWLER, 2005). A Figura 5 mostra o diagrama focado nas entidades do domínio, que correspondem diretamente às classes implementadas no arquivo `app/models.py` e às tabelas do DER.

Figura 5 – Diagrama de classes das entidades do PetAgenda



Fonte: elaborado pelo autor (2026).

As associações indicam a multiplicidade dos relacionamentos: um Cliente possui zero ou muitos objetos Pet, e um Pet, um Servico e um Usuario podem estar associados a zero ou muitos objetos Agendamento. As enumerações Perfil, Especie,

Porte e StatusAgendamento restringem os valores possíveis desses atributos. Entre as operações, destacam-se as da classe Agendamento: calcular_fim, que obtém o horário de término a partir da duração do serviço; existe_conflito, que verifica a sobreposição de horários; e pode_alterar, concluir e cancelar, que controlam o ciclo de vida do agendamento (RN07). Na classe Usuario, as operações definir_senha e verificar_senha garantem que a senha seja armazenada apenas na forma de *hash* (RNF01).

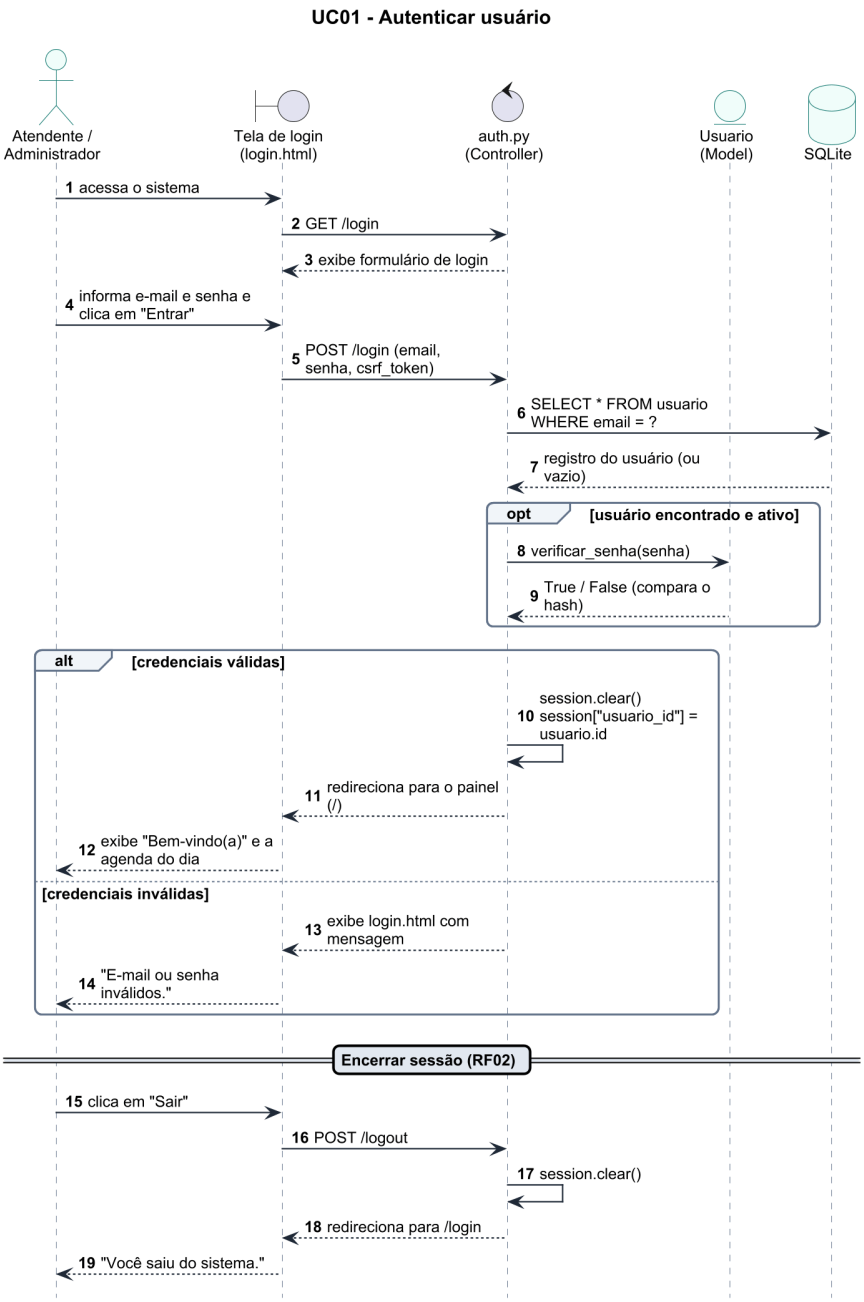
2.9 DIAGRAMAS DE SEQUÊNCIA

O diagrama de sequência mostra a ordem temporal das mensagens trocadas entre os objetos que participam de um caso de uso (GUEDES, 2018). Nos diagramas do PetAgenda, os participantes seguem a arquitetura MVC: o ator, a tela (fronteira ou *boundary*), o controlador (*control*), o módulo de validações, a entidade (*entity*) e o banco de dados SQLite. Os fragmentos combinados **alt** representam as operações do CRUD e os fluxos alternativos, e o fragmento **opt** representa um trecho opcional. As mensagens são numeradas para facilitar a leitura, e as subseções a seguir apresentam os diagramas de cada caso de uso.

2.9.1 UC01 – Autenticar usuário

A Figura 6 mostra a validação das credenciais: o controlador busca o usuário pelo e-mail, compara o *hash* da senha e, em caso de sucesso, cria a sessão; o encerramento da sessão (RF02) aparece ao final do diagrama.

Figura 6 – Diagrama de sequência do UC01 – Autenticar usuário

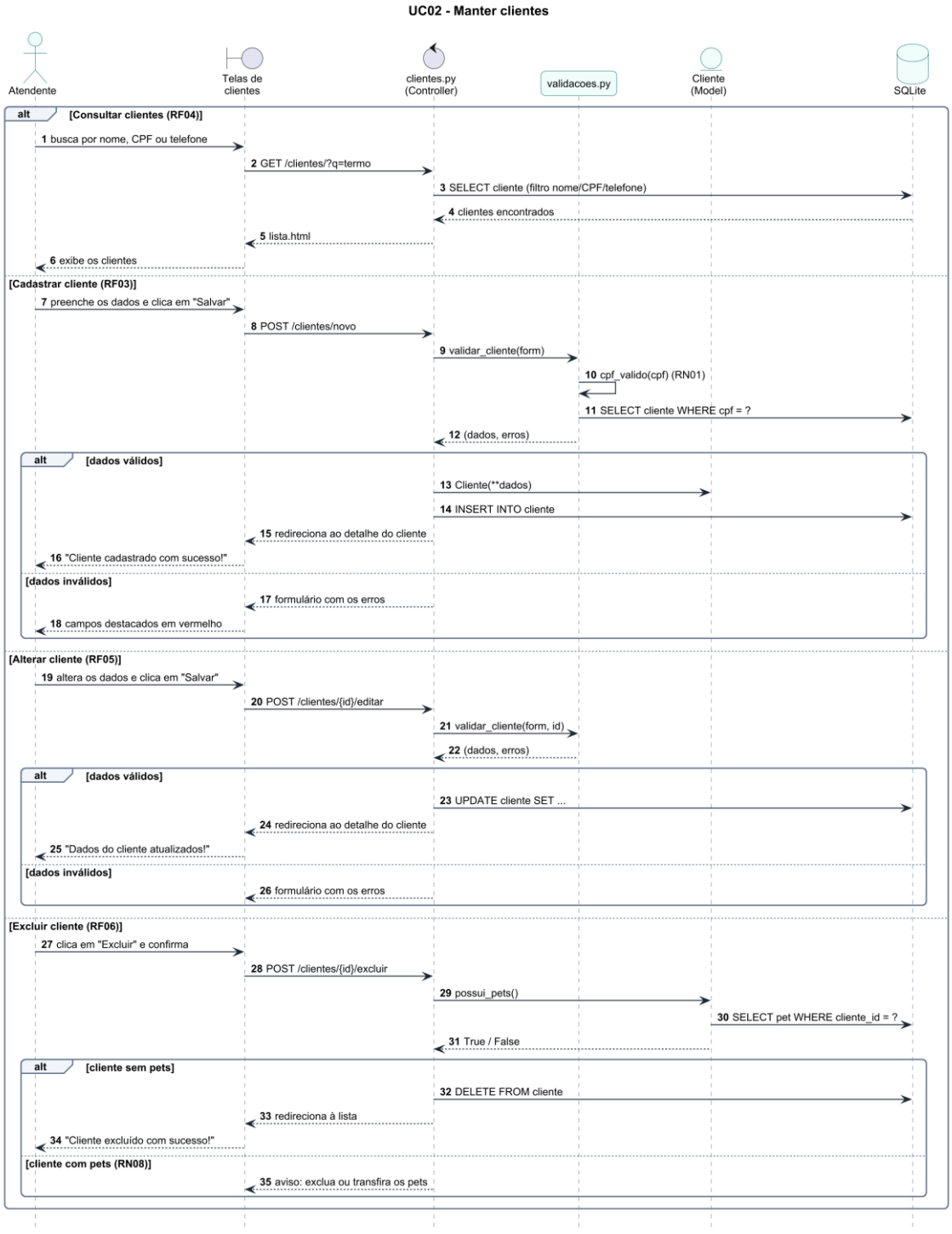


Fonte: elaborado pelo autor (2026).

2.9.2 UC02 – Manter clientes

A Figura 7 detalha as quatro operações do CRUD de clientes, incluindo a validação do CPF (RN01) e o bloqueio da exclusão de clientes que possuem pets (RN08).

Figura 7 – Diagrama de sequência do UC02 – Manter clientes



Fonte: elaborado pelo autor (2026).

2.9.3 UC03 – Manter pets

A Figura 8 apresenta o CRUD de pets, com a verificação de que o tutor informado existe (RN02) e o bloqueio da exclusão de pets com agendamentos (RN08).

Figura 8 – Diagrama de sequência do UC03 – Manter pets

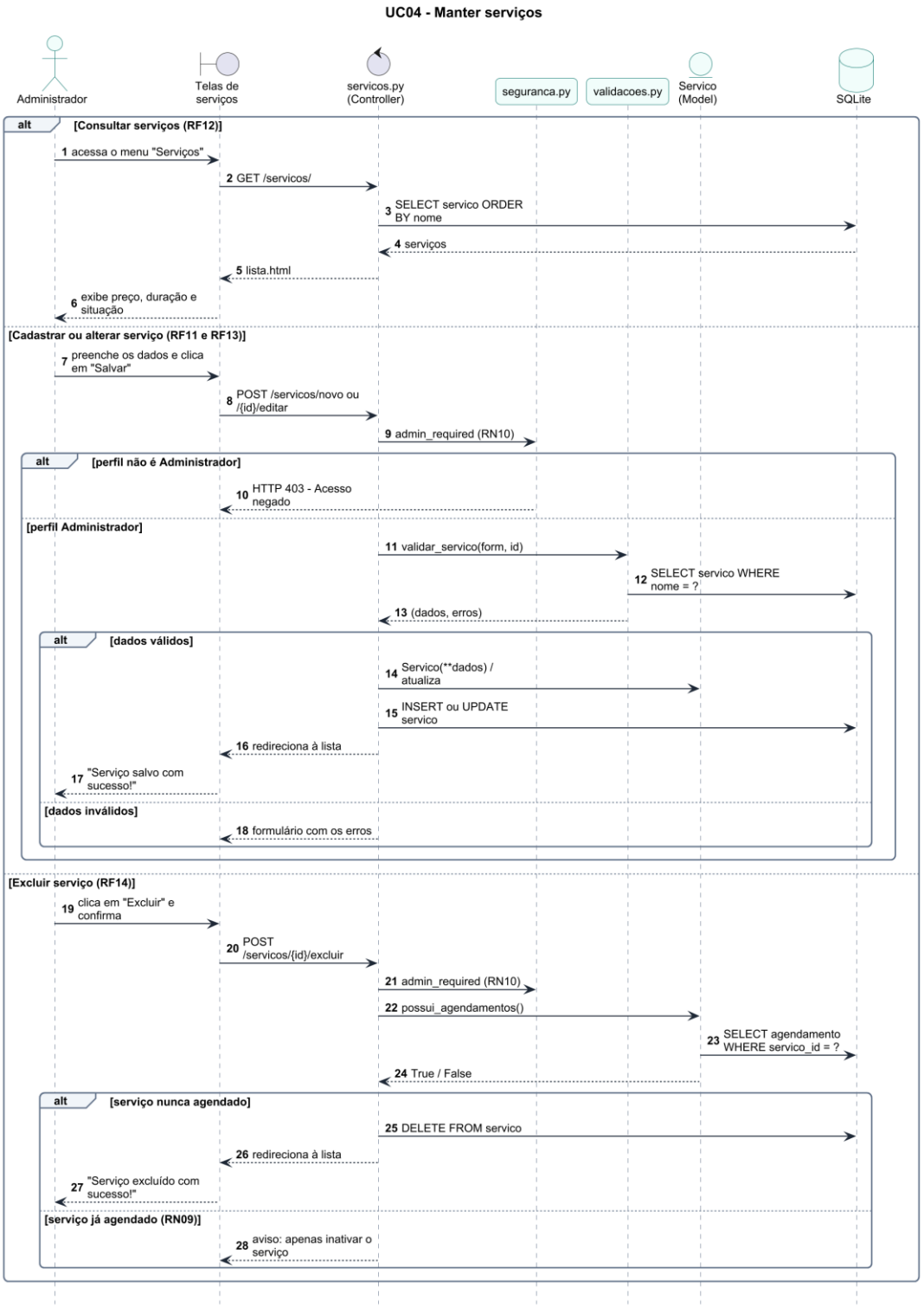


Fonte: elaborado pelo autor (2026).

2.9.4 UC04 – Manter serviços

A Figura 9 mostra o CRUD de serviços, no qual o módulo de segurança verifica se o usuário possui o perfil Administrador (RN10) antes de qualquer alteração.

Figura 9 – Diagrama de sequência do UC04 – Manter serviços

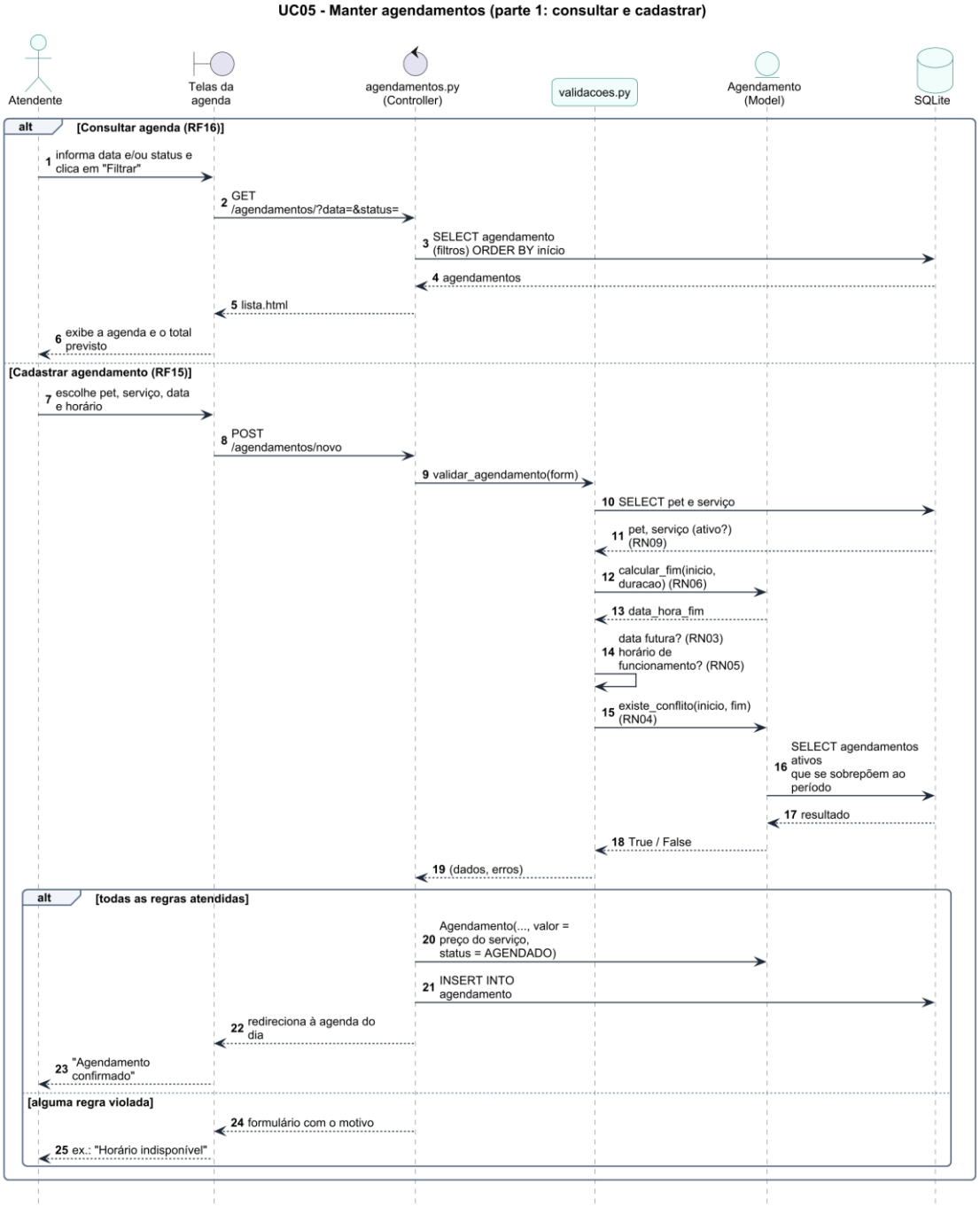


Fonte: elaborado pelo autor (2026).

2.9.5 UC05 – Manter agendamentos

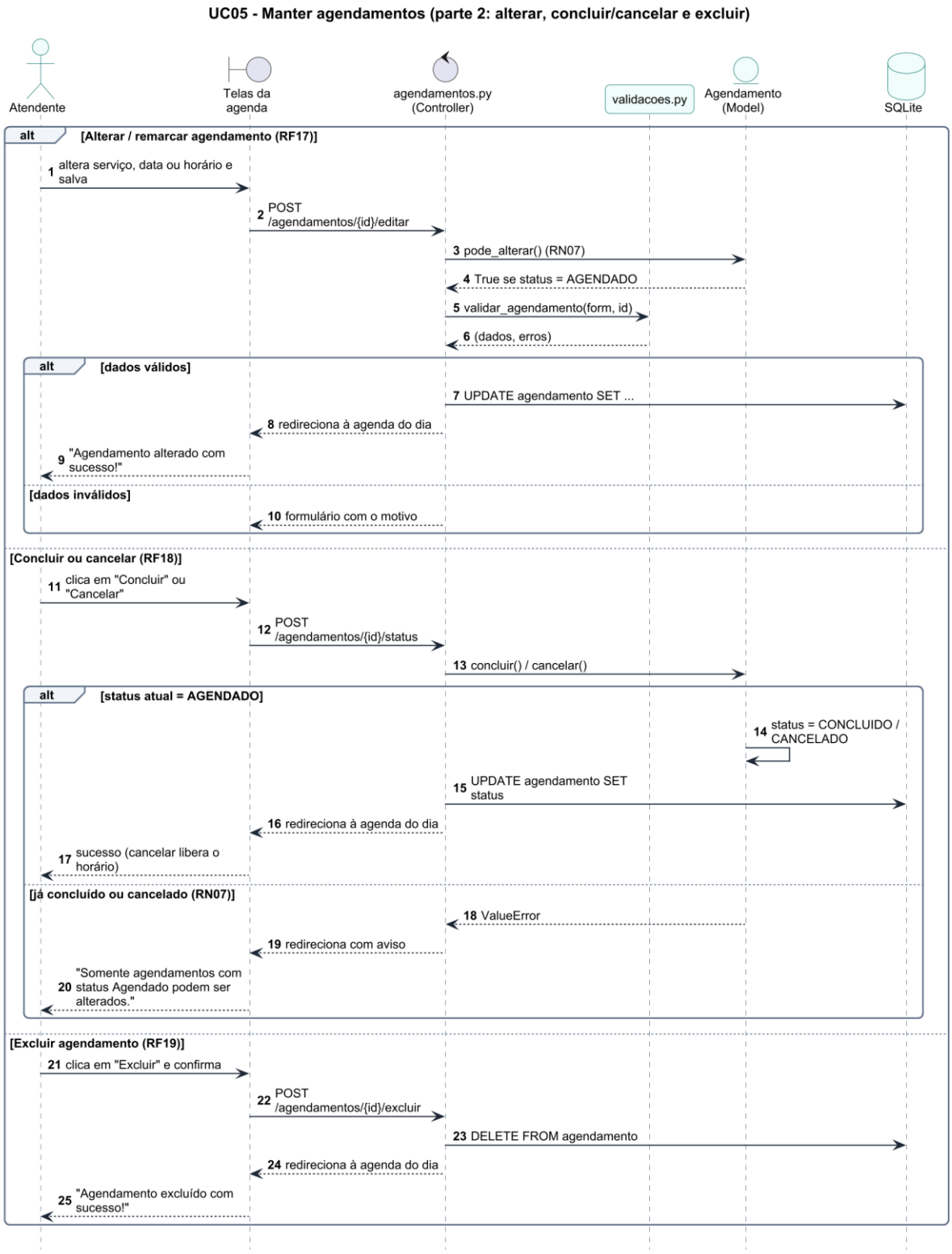
Por ser o caso de uso mais complexo, o diagrama do UC05 foi dividido em duas partes. A Figura 10 apresenta a consulta da agenda e o cadastro de um agendamento, com a aplicação das regras RN03, RN04, RN05, RN06 e RN09; a Figura 11 apresenta a remarcação, a conclusão ou o cancelamento e a exclusão de agendamentos (RN07).

Figura 10 – Diagrama de sequência do UC05 – Manter agendamentos (parte 1)



Fonte: elaborado pelo autor (2026).

Figura 11 – Diagrama de sequência do UC05 – Manter agendamentos (parte 2)

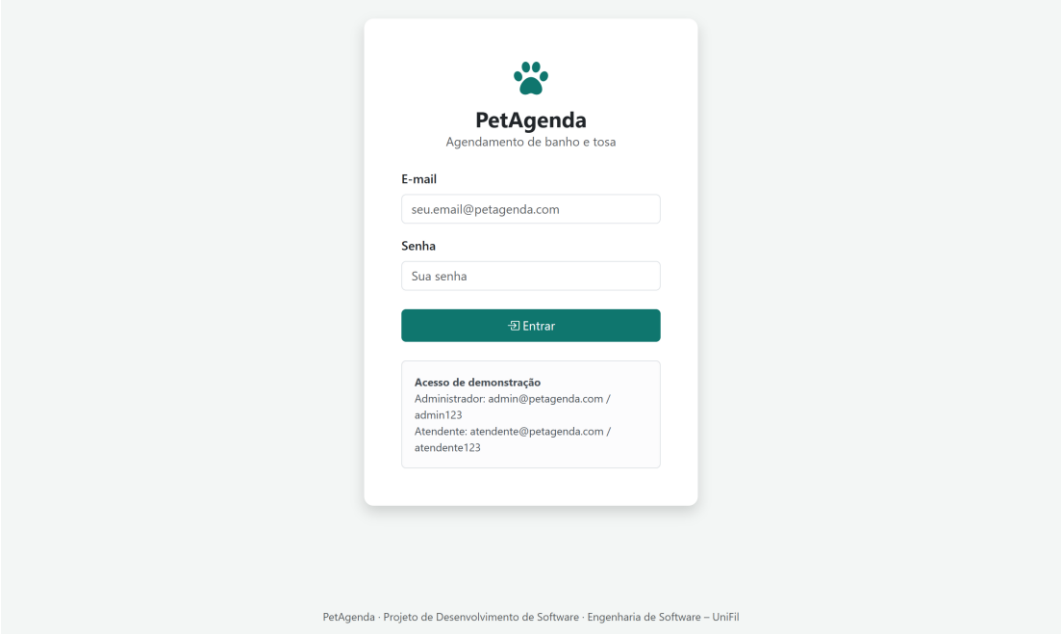


Fonte: elaborado pelo autor (2026).

2.10 PROTÓTIPOS DAS INTERFACES

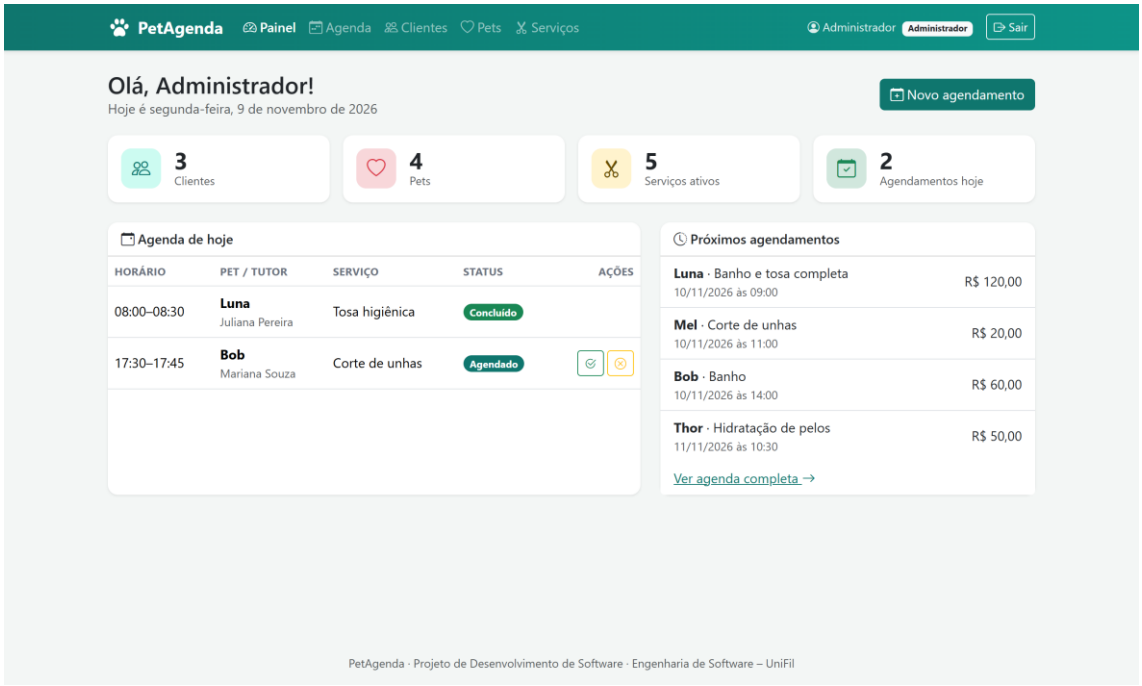
As interfaces foram desenvolvidas com HTML5, CSS3 e o *framework* Bootstrap 5 (BOOTSTRAP, 2024), o que garante um *layout* responsivo e consistente em todas as telas (RNF04). As figuras a seguir mostram as principais telas do sistema implementado: a tela de login (Figura 12), o painel inicial com a agenda do dia (Figura 13), a agenda com filtros e ações por status (Figura 14) e o cadastro de agendamento exibindo a validação de conflito de horário (Figura 15).

Figura 12 – Tela de login



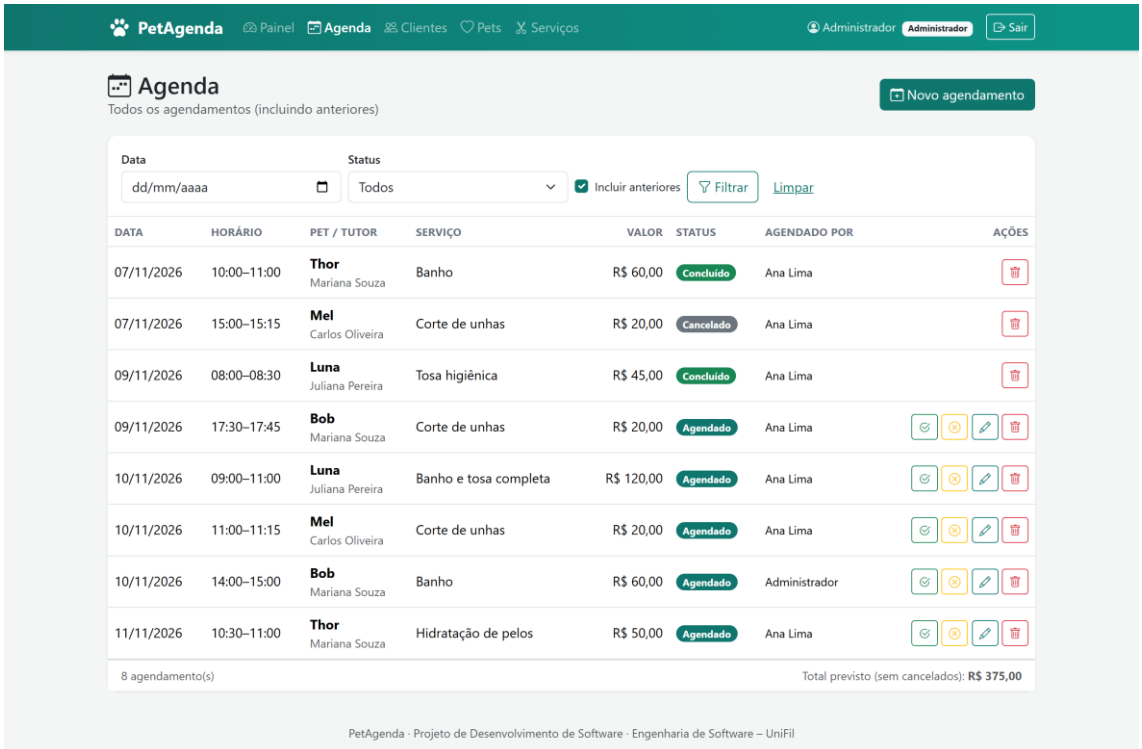
Fonte: elaborado pelo autor (2026).

Figura 13 – Painel inicial com a agenda do dia



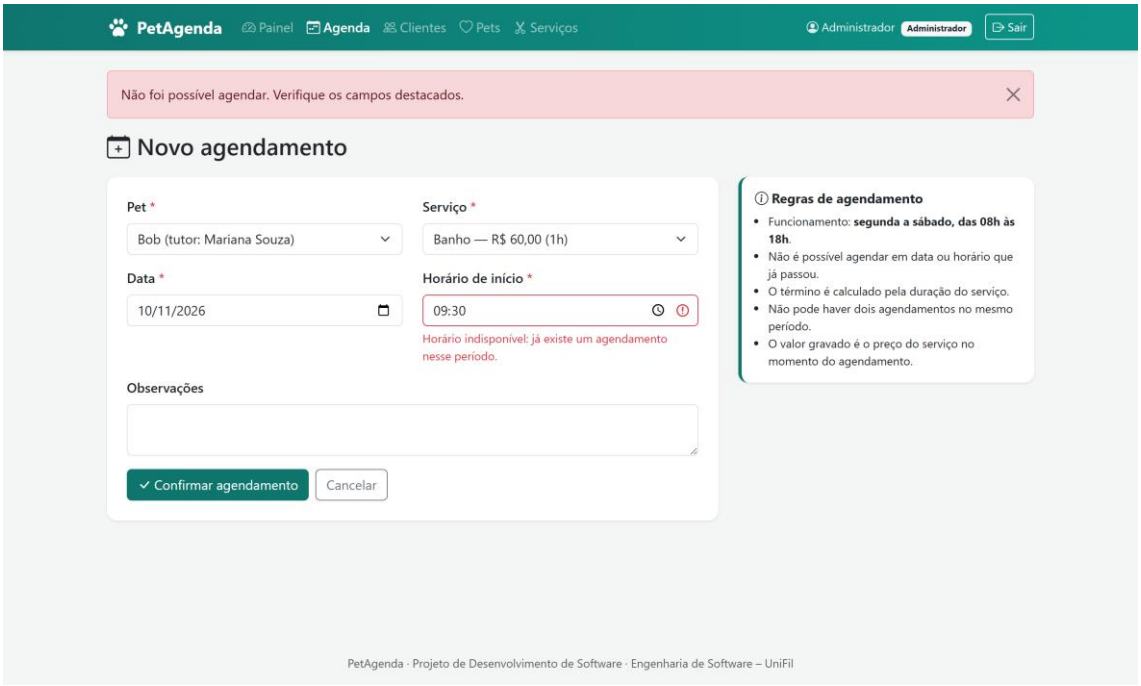
Fonte: elaborado pelo autor (2026).

Figura 14 – Agenda com filtros por data e status



Fonte: elaborado pelo autor (2026).

Figura 15 – Cadastro de agendamento com a validação de conflito de horário (RN04)



Fonte: elaborado pelo autor (2026).

2.11 IMPLEMENTAÇÃO E REPOSITÓRIO

A implementação segue a documentação: cada caso de uso corresponde a um *blueprint* (controlador) com as suas rotas, cada classe do diagrama de classes corresponde a um modelo do SQLAlchemy e cada tabela do DER é criada automaticamente a partir desses modelos. O Quadro 11 apresenta a rastreabilidade entre os casos de uso e os artefatos de código.

Quadro 11 – Rastreabilidade entre os casos de uso e a implementação

Caso de uso	Controlador (rotas)	Telas (templates)	Testes automatizados
UC01 – Autenticar usuário	app/routes/auth.py	auth/login.html	tests/test_auth.py
UC02 – Manter clientes	app/routes/clientes.py	clientes/lista, form e detalhe	tests/test_clientes.py
UC03 – Manter pets	app/routes/pets.py	pets/lista e form	tests/test_pets.py
UC04 – Manter serviços	app/routes/servicos.py	servicos/lista e form	tests/test_servicos.py

Caso de uso	Controlador (rotas)	Telas (templates)	Testes automatizados
UC05 – Manter agendamentos	app/routes/agendamentos.py app/routes/main.py (painel)	agendamentos/lista e form; painel.html	tests/test_agendamentos.py

Fonte: elaborado pelo autor (2026).

As regras de negócio e as operações de CRUD são verificadas por 40 testes automatizados escritos com o pytest (PYTEST, 2025) e executados com um banco de dados em memória. O código-fonte está versionado no GitHub, no endereço https://github.com/Lucas-Sambati/petagenda. Para executar o sistema localmente, são necessários o Python 3.11 ou superior e os comandos a seguir; depois, basta acessar o endereço http://127.0.0.1:5000 com o usuário admin@petagenda.com e a senha admin123.

```
git clone https://github.com/Lucas-Sambati/petagenda.git
cd petagenda
python -m venv .venv
.venv\Scripts\activate          (no Linux/macOS: source .venv/bin/activate)
pip install -r requirements.txt
python run.py
```

3 CONCLUSÃO

Este trabalho apresentou a documentação e a implementação do PetAgenda, um sistema web para o agendamento de serviços de banho e tosa em pet shops de pequeno porte. Foram especificados 20 requisitos funcionais, 10 requisitos não funcionais e 10 regras de negócio, modelados os cinco casos de uso do sistema e elaborados o DER, o diagrama de atividade, o diagrama de classes e os diagramas de sequência de cada caso de uso. O sistema implementado realiza todas as operações de CRUD com persistência em banco de dados SQLite e atende aos requisitos levantados.

O principal aprendizado foi perceber o quanto a etapa de documentação orienta e simplifica a implementação. Ao escrever as regras de negócio, surgiram decisões que não eram evidentes no início, como impedir conflitos de horário, gravar o preço do serviço no momento do agendamento para preservar o histórico e bloquear a exclusão de registros que possuem dependentes. Se essas questões fossem descobertas apenas durante a programação, exigiriam retrabalho no banco de dados e no código.

A modelagem com a UML também mostrou a utilidade de cada diagrama: o diagrama de casos de uso delimitou o escopo e acabou definindo a própria organização do código, com um controlador para cada caso de uso; o diagrama de atividade ajudou a entender o processo do pet shop antes de pensar nas telas; e os diagramas de sequência deixaram clara a responsabilidade de cada camada do MVC, evitando que regras de negócio ficassem espalhadas pelas rotas. A correspondência direta entre o diagrama de classes, as classes do ORM e as tabelas do DER evidenciou a importância de manter a documentação e o código consistentes.

No projeto do banco de dados, foi possível aplicar na prática os conceitos de chaves primárias e estrangeiras, cardinalidade e restrições de integridade, além de aprender que o SQLite exige a ativação explícita das chaves estrangeiras. Na implementação, foram consolidados conhecimentos sobre o Flask, o mapeamento objeto-relacional, a segurança de aplicações web (armazenamento de senhas com *hash*, controle de acesso por perfil e proteção contra CSRF), os testes automatizados e o versionamento com Git e GitHub.

Como trabalhos futuros, propõem-se: permitir mais de uma estação de atendimento simultânea; enviar lembretes automáticos aos tutores; oferecer o agendamento on-line pelos próprios clientes; gerar relatórios de faturamento; e publicar o sistema em um servidor na nuvem, com um banco de dados como o PostgreSQL.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. **UML**: guia do usuário. 2. ed. Rio de Janeiro: Elsevier, 2012.

BOOTSTRAP. **Bootstrap documentation**: v5.3. [S. l.]: Bootstrap Team, 2024. Disponível em: https://getbootstrap.com/docs/5.3/. Acesso em: 9 nov. 2026.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson Education do Brasil, 2018.

FOWLER, Martin. **UML essencial**: um breve guia para a linguagem-padrão de modelagem de objetos. 3. ed. Porto Alegre: Bookman, 2005.

GUEDES, Gilleanes T. A. **UML 2**: uma abordagem prática. 3. ed. São Paulo: Novatec, 2018.

HEUSER, Carlos Alberto. **Projeto de banco de dados**. 6. ed. Porto Alegre: Bookman, 2009.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010**: systems and software engineering: systems and software quality requirements and evaluation (SQuaRE): product quality model. Geneva: ISO, 2023.

OBJECT MANAGEMENT GROUP. **OMG Unified Modeling Language (OMG UML)**: version 2.5.1. Needham: OMG, 2017. Disponível em: https://www.omg.org/spec/UML/2.5.1. Acesso em: 9 nov. 2026.

PALLETS. **Flask documentation**: version 3.1.x. [S. l.]: Pallets, 2025. Disponível em: https://flask.palletsprojects.com/. Acesso em: 9 nov. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.

PYTEST. **pytest documentation**. [S. l.]: pytest-dev, 2025. Disponível em: https://docs.pytest.org/. Acesso em: 9 nov. 2026.

PYTHON SOFTWARE FOUNDATION. **Python 3.13 documentation**. [S. l.]: PSF, 2025. Disponível em: https://docs.python.org/3.13/. Acesso em: 9 nov. 2026.

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson Education do Brasil, 2018.

SQLALCHEMY. **SQLAlchemy documentation**. [S. l.]: SQLAlchemy authors and contributors, 2025. Disponível em: https://docs.sqlalchemy.org/. Acesso em: 9 nov. 2026.

SQLITE. **SQLite documentation**. [S. l.]: SQLite Consortium, 2025. Disponível em: https://www.sqlite.org/docs.html. Acesso em: 9 nov. 2026.

APÊNDICE A – SCRIPT DE CRIAÇÃO DO BANCO DE DADOS

O script a seguir corresponde ao modelo físico do banco de dados do PetAgenda. Ele foi gerado a partir das classes de entidade do sistema e está disponível no arquivo `docs/banco/schema.sql` do repositório.

```
-- Modelo físico do banco de dados do PetAgenda (SQLite)
-- Gerado a partir das classes do arquivo app/models.py (SQLAlchemy).
-- As tabelas são criadas automaticamente ao executar a aplicação.

PRAGMA foreign_keys = ON;

CREATE TABLE cliente (
    id INTEGER NOT NULL,
    nome VARCHAR(100) NOT NULL,
    cpf VARCHAR(11) NOT NULL,
    telefone VARCHAR(11) NOT NULL,
    email VARCHAR(120),
    endereco VARCHAR(200),
    criado_em DATETIME NOT NULL,
    PRIMARY KEY (id),
    UNIQUE (cpf)
);

CREATE TABLE servico (
    id INTEGER NOT NULL,
    nome VARCHAR(80) NOT NULL,
    descricao VARCHAR(255),
    preco NUMERIC(10, 2) NOT NULL,
    duracao_minutos INTEGER NOT NULL,
    ativo BOOLEAN NOT NULL,
    PRIMARY KEY (id),
    UNIQUE (nome)
);

CREATE TABLE usuario (
    id INTEGER NOT NULL,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(120) NOT NULL,
    senha_hash VARCHAR(255) NOT NULL,
    perfil VARCHAR(20) NOT NULL,
    ativo BOOLEAN NOT NULL,
    criado_em DATETIME NOT NULL,
    PRIMARY KEY (id),
    UNIQUE (email),
    CONSTRAINT perfil CHECK (perfil IN ('ADMIN', 'ATENDENTE'))
);

CREATE TABLE pet (
    id INTEGER NOT NULL,
    cliente_id INTEGER NOT NULL,
    nome VARCHAR(60) NOT NULL,
    especie VARCHAR(20) NOT NULL,
    raca VARCHAR(60),
    porte VARCHAR(20) NOT NULL,
    data_nascimento DATE,
    observacoes TEXT,
    criado_em DATETIME NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY(cliente_id) REFERENCES cliente (id) ON DELETE RESTRICT,
    CONSTRAINT especie CHECK (especie IN ('CAO', 'GATO', 'OUTRO')),
    CONSTRAINT porte CHECK (porte IN ('PEQUENO', 'MEDIO', 'GRANDE'))
);
CREATE INDEX ix_pet_cliente_id ON pet (cliente_id);
```

```
CREATE TABLE agendamento (
    id INTEGER NOT NULL,
    pet_id INTEGER NOT NULL,
    servico_id INTEGER NOT NULL,
    usuario_id INTEGER NOT NULL,
    data_hora_inicio DATETIME NOT NULL,
    data_hora_fim DATETIME NOT NULL,
    valor NUMERIC(10, 2) NOT NULL,
    status VARCHAR(20) NOT NULL,
    observacoes TEXT,
    criado_em DATETIME NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY(pet_id) REFERENCES pet (id) ON DELETE RESTRICT,
    FOREIGN KEY(servico_id) REFERENCES servico (id) ON DELETE RESTRICT,
    FOREIGN KEY(usuario_id) REFERENCES usuario (id) ON DELETE RESTRICT,
    CONSTRAINT statusagendamento CHECK (status IN ('AGENDADO', 'CONCLUIDO', 'CANCELADO'))
);
CREATE INDEX ix_agendamento_data_hora_inicio ON agendamento (data_hora_inicio);
CREATE INDEX ix_agendamento_pet_id ON agendamento (pet_id);
CREATE INDEX ix_agendamento_servico_id ON agendamento (servico_id);
CREATE INDEX ix_agendamento_usuario_id ON agendamento (usuario_id);
```